

Contents

Factorial Experiment Roadmap	1
Overview: $2 \times 3 \times 3 = 18$ Experimental Conditions	1
Experimental Design Summary	1
Factor 1: Recognition Models	2
Level 1a: GRU Recognition Model	2
Level 1b: Contrastive Recognition Model	2
Factor 2: Dream Generation Strategies	2
Level 2a: Standard Dreaming	2
Level 2b: Balanced Dreaming	3
Level 2c: Contrastive Dreaming	3
Factor 3: Primitive Libraries	3
Level 3a: Lean Primitives (67 primitives)	3
Level 3b: Lean+Fold Primitives (78 primitives)	4
Level 3c: Minimal Primitives (56 primitives)	4
Complete Condition Matrix	5
Shared Configuration Across All Conditions	6
Enumeration Algorithm: TopDownEnumerator	6
Recognition Guidance Integration	6
Wake-Sleep Iteration Structure	6
Task Set	6
Expected Outcomes by Condition	7
Predicted Ranking (tasks solved):	7
Timing Estimate	7
Launch Command	7
Output Files	8

Factorial Experiment Roadmap

Overview: $2 \times 3 \times 3 = 18$ Experimental Conditions

This document provides a detailed breakdown of all 18 conditions in the factorial experiment, including the specific algorithms, configurations, and expected behaviors for each factor level.

Experimental Design Summary

Factor	Levels	Description
Recognition Model	2	GRU vs Contrastive
Dream Strategy	3	Standard vs Balanced vs Contrastive
Primitive Library	3	Lean vs Lean+Fold vs Minimal

Total Conditions: $2 \times 3 \times 3 = 18$ conditions

Factor 1: Recognition Models

Level 1a: GRU Recognition Model

File: dreamcoder_core/neural_recognition.py

Architecture: - Bidirectional GRU encoder for hand examples - Attention mechanism over example encodings - Per-primitive output heads (multi-label classification)

Input Processing: 1. Each example (hand, bool) encoded via card embeddings 2. Card features: suit (4-dim), rank (13-dim), color (2-dim), position 3. GRU processes sequence of cards 4. Attention aggregates example representations

Output: - Log-probability distribution over primitives - Used to reweight grammar productions for guided enumeration

Training: - Binary cross-entropy loss per primitive - Target: primitives used in solution program
- Epochs: 5 per wake-sleep iteration

Key Properties: - Sequential processing (respects card order) - Attention allows focusing on discriminative examples - ~10K parameters (hidden_dim=64)

Level 1b: Contrastive Recognition Model

File: dreamcoder_core/contrastive_recognition.py

Architecture: - Set-based card encoder (order-invariant) - Contrastive learning objective - Learned embeddings for tasks and primitives

Input Processing: 1. Cards encoded independently (no sequence structure) 2. Set pooling (sum/mean) over card embeddings 3. Example-level aggregation via attention

Output: - Same format as GRU: log-probs over primitives - Optimized for discriminating similar tasks

Training: - Contrastive loss: similar tasks should predict similar primitives - Uses positive pairs (same solution structure) and negatives (different) - Epochs: 5 per wake-sleep iteration

Key Properties: - Permutation-invariant over cards (set semantics) - Better at learning task similarity structure - Potentially faster convergence on related tasks

Factor 2: Dream Generation Strategies

Level 2a: Standard Dreaming

File: dreamcoder_core/contrastive_dreaming.py (strategy='standard')

Algorithm: 1. Sample random program from grammar (weighted by production probabilities) 2. Execute program on randomly sampled hands 3. Record input-output pairs as dream examples 4. **No class balancing** - natural distribution emerges

Example Distribution: - If program tends to return True 90% of time, dreams are 90% positive
- May learn programs with extreme base rates

Use Case: - Matches original DreamCoder implementation - Baseline for comparison

Expected Behavior: - Fast to generate - May produce trivial programs (always true/false) - Recognition model sees biased training distribution

Level 2b: Balanced Dreaming

File: dreamcoder_core/contrastive_dreaming.py (strategy='balanced')

Algorithm: 1. Sample random program from grammar 2. Execute on hands until we have **equal positives and negatives** 3. Use rejection sampling to balance classes 4. Negative examples are **random** hands that fail the predicate

Example Distribution: - Always 50% True, 50% False - Negatives are random (not near-misses)

Use Case: - Removes class imbalance bias - Standard approach in supervised learning

Expected Behavior: - More informative training signal - Recognition model learns discriminative features - May take longer to generate (rejection sampling)

Level 2c: Contrastive Dreaming

File: dreamcoder_core/contrastive_dreaming.py (strategy='contrastive')

Algorithm: 1. Sample program from grammar 2. Generate positive examples (hands where program returns True) 3. Generate **near-miss negatives**: - Take a positive hand - Swap 1-2 cards to break the predicate - Verify the modified hand is now False 4. Final examples: 50% positive, 50% near-miss negative

Near-Miss Generation:

```
positive_hand = [Ah, Kh, Qh, Jh, Th] # flush
swap one card with random card
negative_hand = [Ah, Kh, Qh, Jh, 2s] # not a flush (near-miss)
```

Example Distribution: - 50% True, 50% False - Negatives are **structurally similar** to positives

Use Case: - Inspired by cognitive science (contrast improves category learning) - Should help recognition model learn decision boundaries

Expected Behavior: - Most informative training signal - Recognition model learns what features matter - Slowest to generate (requires perturbation + verification)

Factor 3: Primitive Libraries

Level 3a: Lean Primitives (67 primitives)

File: dreamcoder_core/lean_primitives.py

Design Philosophy: "Cognitive Realism" - High-level primitives matching human conceptual vocabulary - Direct queries without decomposition - Domain-specific aggregates

Primitive Categories: | Category | Count | Examples | |-----|-----|-----| | Constants | 21 | true, false, 1-10, suits, ranks | | Card Accessors | 4 | get_suit, get_rank, rank_val, get_color | | Position Ops | 5 | length, first, last, nth, index | | List Slicing | 7 | take, drop, first_half, adjacent_pairs | | Direct Queries | 9 | has_suit, all_same_suit, count_color, count_rank | | Aggregates | 3 | sum_ranks, max_rank, min_rank | | Comparisons | 6 | eq, lt, gt, le, ge, neq | | Boolean Ops | 4 | and, or, not, if_then_else | | Higher-Order | 5 | map, filter, all, any, unique | | Arithmetic | 3 | +, -, * |

Example Programs:

```
all_same_suit(hand)                # Direct query
eq(count_color(hand, Red), 3)      # Using aggregates
all(lambda c: eq(get_suit(c), Hearts), hand) # Composed
```

Expected Behavior: - Fast initial learning (primitives match task concepts) - Less room for compression (already high-level) - Lower search depth needed

Level 3b: Lean+Fold Primitives (78 primitives)

File: experiments/primitive_variants.py

Design Philosophy: Universal Computation - Everything from Lean, plus... - DreamCoder's fold/cons primitives for general iteration - Tuple operations from Yang & Piantadosi (2022)

Additional Primitives: | Category | Count | Examples | |-----|-----|-----| | Fold Operations | 6 | fold, foldr, cons, empty, tail, is_empty | | Bool Aggregators | 2 | all_true, any_true | | Pair Operations | 3 | pair, fst, snd |

Fold Semantics:

```
fold : (acc -> elem -> acc) -> acc -> list -> acc
fold (+) 0 [1,2,3,4,5] = 15 # sum
fold (max) 0 ranks = max_rank
```

Example Programs:

```
fold (lambda acc c: and acc (eq (get_suit c) Hearts)) true hand # all hearts
foldr (lambda c acc: cons (get_rank c) acc) empty hand # extract ranks
```

Expected Behavior: - More expressive (can encode any list computation) - Larger search space (more primitives) - May discover novel abstractions via fold compositions - Slower enumeration (more choices at each step)

Level 3c: Minimal Primitives (56 primitives)

File: experiments/primitive_variants.py

Design Philosophy: Force Abstraction Learning - Remove high-level convenience primitives - Model must DISCOVER patterns through compression - Tests library learning capability

Removed from Lean: | Removed Category | Count | Examples | |-----|-----|-----|
 ---| | Direct Queries | 9 | `has_suit`, `all_same_suit`, `count_color`, etc. | | Aggregates | 3 |
`sum_ranks`, `max_rank`, `min_rank` | | Convenience Slicing | 4 | `first_half`, `second_half`, `half_len`,
`adjacent_pairs` | | Unique | 1 | `unique` |

Added for Compensation: | Added | Count | Reason | |-----|-----|-----| | Fold Operations | 6
 | Needed to express removed aggregates |

Example: Must-Learn Patterns:

```
# Without all_same_suit, must compose:
all_same_suit(h) = eq(1, length(unique(map(get_suit, h))))
```

```
# Without sum_ranks, must compose:
sum_ranks(h) = fold(+, 0, map(rank_val, h))
```

These become library inventions after compression!

Expected Behavior: - Slowest initial learning (programs are longer) - Richest compression potential (many shared subprograms) - Tests core DreamCoder thesis: library learning - May discover abstractions not in Lean

Complete Condition Matrix

#	Recognition	Dreams	Primitives	Condition Name
1	GRU	Standard	Lean	<code>gru_standard_lean</code>
2	GRU	Standard	Lean+Fold	<code>gru_standard_lean_plus_fold</code>
3	GRU	Standard	Minimal	<code>gru_standard_minimal</code>
4	GRU	Balanced	Lean	<code>gru_balanced_lean</code>
5	GRU	Balanced	Lean+Fold	<code>gru_balanced_lean_plus_fold</code>
6	GRU	Balanced	Minimal	<code>gru_balanced_minimal</code>
7	GRU	Contrastive	Lean	<code>gru_contrastive_lean</code>
8	GRU	Contrastive	Lean+Fold	<code>gru_contrastive_lean_plus_fold</code>
9	GRU	Contrastive	Minimal	<code>gru_contrastive_minimal</code>
10	Contrastive	Standard	Lean	<code>contrastive_standard_lean</code>
11	Contrastive	Standard	Lean+Fold	<code>contrastive_standard_lean_plus_fold</code>
12	Contrastive	Standard	Minimal	<code>contrastive_standard_minimal</code>
13	Contrastive	Balanced	Lean	<code>contrastive_balanced_lean</code>
14	Contrastive	Balanced	Lean+Fold	<code>contrastive_balanced_lean_plus_fold</code>
15	Contrastive	Balanced	Minimal	<code>contrastive_balanced_minimal</code>
16	Contrastive	Contrastive	Lean	<code>contrastive_contrastive_lean</code>
17	Contrastive	Contrastive	Lean+Fold	<code>contrastive_contrastive_lean_plus_f</code>
18	Contrastive	Contrastive	Minimal	<code>contrastive_contrastive_minimal</code>

Shared Configuration Across All Conditions

Enumeration Algorithm: TopDownEnumerator

Key Properties: - Best-first search using priority queue - Cost = $-\log_probability$ (lower is better) - Programs enumerated in order of decreasing probability - Neural guidance directly affects enumeration order

Progressive Budget (per iteration):

Iteration	Budget	Max Depth	Timeout
1	50,000	6	45s
2	65,000	7	60s
3	80,000	8	75s
4	95,000	9	90s
5	100,000	10	120s

Recognition Guidance Integration

When: Iterations 2+ (iteration 1 uses base grammar)

How: 1. Recognition model predicts log-probs for each primitive given task 2. Blend with base grammar weights: $new_lp = (1 - \alpha) * base_lp + \alpha * predicted_lp$ 3. Alpha increases over iterations: 0.3 $\rightarrow$ 0.8 4. Grammar passed to worker process as primitive log-probs dict 5. Worker rebuilds grammar and applies weights before enumeration

Wake-Sleep Iteration Structure

For each iteration:

1. WAKE: Parallel enumeration for all tasks
 - 6 worker processes
 - TopDownEnumerator per task
 - Recognition guidance (if iteration > 1)
2. SLEEP-COMPRESS: Extract abstractions
 - Find common subtrees in solutions
 - Add as invented primitives
 - Max 3 new inventions per iteration
3. SLEEP-RECOGNIZE: Train recognition model
 - On solved tasks and their solutions
 - 5 epochs
 - Update primitive embeddings
4. SLEEP-DREAM: Generate synthetic tasks
 - 50 dreams per iteration
 - 10 examples per dream
 - Strategy-specific generation

Task Set

Source: rules/catalogue.py (experimental rules)

Sampling: - 6-card hands (not 5) - Balanced sampling: 50% True, 50% False - 50 training examples, 20 holdout per task - Holdout verification required for solutions

Rule Families: 1. Suit-based (has_suit, all_same, uniform_color) 2. Rank-based (has_rank, pair, sequence) 3. Composite (combination rules) 4. Counting (exactly N of X) 5. Ordering (sorted, adjacent)

Expected Outcomes by Condition

Predicted Ranking (tasks solved):

Fastest Learning (most tasks solved early): 1. *_balanced_lean - Best primitives + good training signal 2. *_contrastive_lean - Best primitives + optimal training 3. *_standard_lean - Best primitives, baseline training

Slowest but Richest: - *_*_minimal - Must discover abstractions, but best library growth

Recognition Model Impact: - Contrastive recognition may excel on related tasks (transfer) - GRU may be more robust to noisy examples

Dream Strategy Impact: - Contrastive dreams should accelerate recognition learning - Standard dreams may produce degenerate programs

Timing Estimate

Component	Time per Condition
Enumeration (4 iters x 22 tasks)	~20-30 min
Recognition training	~5 min
Compression	~2 min
Dreaming	~3 min
Total per condition	~30-40 min

Full experiment: 18 conditions x 35 min = ~10-12 hours

Launch Command

```
# Full experiment with caffeinate (prevent sleep)
cd /Users/cankonuk/Documents/self-explanations-project/card-games-modelling/src
nohup caffeinate -d -i -s python3 experiments/run_factorial_experiment.py \
    --max-iterations 4 \
    --results-dir results_factorial \
    > factorial_experiment.log 2>&1 &

# Monitor progress
tail -f factorial_experiment.log

# Check status
ps aux | grep run_factorial | grep -v grep
```

Output Files

Each condition produces:

```
results_factorial/  
  {recognition}_{dreams}_{primitives}/  
    run_1/  
      results.json           # Iteration-by-iteration metrics  
      final_results.json    # Complete summary  
      model_iter_{N}.pt      # Recognition model checkpoints  
      grammar_info_iter_{N}.json # Learned primitives
```

Document generated for pre-launch review of factorial experiment conditions.